Overall Instructions

Note: This doc has undergone some changes on April 10. Please read the change log below carefully!

💡 Change Log Last Updated: Apr 10, 2026

Date	Type	Description

Apr 10, 2026	**New Universe!**	A new universe is available! The Keystone Mortgage Universe is rolling out with v2.2 tasks.
		
		Highlights:
		• Completely new summary, personas, business functions, scenarios and artifacts
		• New universe fixed date: April 28, 2026
		• Updated Eval tool - download it
		• Different default and non-default tools
		  ○ Default tools (always skip TS): `email*`, `conversations_* (slack)`, `contacts_*`
		  ○ Non-default tools (write TS if agent might skip): `mortgage_los_*`, `stripe_*`, `crm_*`, `filesystem_*`, `quickbooks_*`
		• Different environment and universe IDs:
		  ○ Environment ID: `mortgage-broker-v1`
		  ○ Base Universe ID: `mortgage-broker-01-keystone`
		
		IMPORTANT WARNING: As of April 10, 2026 the FileSystem tool will let you see and explore files, **but the Evals tool is not ready to support it**. You may use FileSystem in-task, but you won't have full coverage from the Evals tool yet. It will be possible to make full use of PDFs, Docs and other artifacts with this tool in the near future.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Date	Type	Description
Apr 07, 2026	**Resource**	Scenario Generation was **disabled** for the time being
Apr 01, 2026	**New Resource**	**Scenario Generation is ready to use**
Mar 30, 2026	**QC Guideline**	**Added updates to the QC Guidelines - Erroneous runs**
Mar 24, 2026	**Fixed date**	**MoveOps universe: Today's date is seen as April 26, 2026 by the agent.**
Mar 23, 2026	**Updated scenarios and personas**	**5 new scenarios and persona updates to explore - let’s make more DIVERSE prompts! Please read through the new scenarios!**

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Date	Type	Description
Mar 23, 2026	**New universe ID**	`mcp-advanced-3-20-moveops-metadata`
Mar 20, 2026	[Example Tasks - v2]()	**Added 2 examples of v2 tasks to show high-quality tasks. Always use them as reference and not as a template to copy.**
Mar 20, 2026	[QC Spec Doc]()	**Updated QC Guidelines to Align with V2 Updates:** 1. Business Function 2. Tool Name mentioned in the prompt 3. V2 Rubrics 4. Oracle Events Completeness 5. And more - see all the **3/20** Updates
Mar 12, 2026	[Rubrics]()	**Update to Rubrics Workflow:** **1.** Start with Outcome rubrics – Write all write actions as Outcome rubrics first. **2.** Outcome categories simplified (from 5 to 3). **3.** New Process Rubric Decision Matrix and Default/Non-Default Tool list added to decide TS and QC rubrics.
Mar 12, 2026	[Diverse Prompt Examples]()	Click on the '[Diverse Prompt Examples]()' to see examples of prompts touching various functions/topics/categories and scenarios in the universe!

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Date	Type	Description
		Also, in the sections for stacking, branching, and step-by-step dependency, we've added diverse types of prompt examples so we can use more tools and servers, like Airtable, CRM,Quickbooks, Linear, etc. Make sure you are diverse from only using Slack and Email (we have a lot of tasks on these)!
Mar 12, 2025	**Tool Call Count**	Instead of checking if one agent run has 40+ tool calls, please make sure the average tool call count is 40+!
Mar 12, 2026	**Failure Rate**	Difficulty target clarified: 0% pass rate is acceptable as long as overall rubrics are high-quality and don't lead to false negatives/invalid model failures.
Mar 7, 2026	**Rubrics**	Removed the 'goal' after the tool in the Tool Selection rubric (e.g., changed 'search_emails to look up client contact details' to 'search_emails').

Your Goal

Create complex tasks for an AI agent operating inside fictional business companies. The agent has access to the company's email, Slack, Linear, calendar, CRM, and other work servers.

You're looking for situations where the AI agent could fail — where it misses something, gets confused, makes a wrong assumption, or doesn't explore deeply enough. A good task is one where the agent needs to dig through many data sources and still gets something wrong.

What this means concretely:

- Your task must cause the agent to make many tool calls (target: 40+) across multiple services.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

- The task should involve investigation and action — the agent reads data from several sources, reasons about it, then takes actions (sends emails, creates issues, books flights, etc.)
- The agent must fail on at least some rubric criteria. If the agent solves your entire task perfectly, it's not hard enough. Iterate.

DISCLAIMER: AVERAGE TOOL CALL COUNT OF ALL AGENT RUNS MUST BE 40+ TOOL CALLS

What you are NOT creating:

- Simple lookups ("How many unread emails do I have?"). Too easy, too few tool calls.
- Single-service tasks ("Search for flights to Miami"). We need cross-service reasoning.
- Command lists ("Step 1: search emails. Step 2: check CRM. Step 3: send email."). The agent should figure out the steps.
- Tasks solvable without tools ("What's the best way to handle an unhappy client?"). We must require actual data.
- Contrived or artificial scenarios that trick the model rather than test real capability.

You will:

- Explore the universes to understand the data and find interesting situations
- Edit the universe to add complexity (encouraged — the base universe has limited natural difficulty)
- Write a prompt — a natural work request that requires broad exploration
- Run the agent
- Write rubrics — during and after the agent run, write specific criteria to grade the agent (see Rubric Guidelines)

Step 1: Understand the Universe

Your Universe

You will receive a base universe. This is your universe to explore, edit, and build tasks from. Other CBs have their own copies. Your edits won't affect theirs. This prevents task overlap and lets you build deep familiarity with your data over time. The first few tasks may take longer as you learn the universe; subsequent tasks should be faster.

The Universe Guide

The Universe Guide

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Read the Universe Guides to understand the companies' background, key personas and their relationships, all pre-built scenarios and storylines, and what data exists across services (email, Slack, Linear, calendar, CRM, etc.)

- `v2.1 tasks and prior`: Move Ops Universe - B2B travel/relocation, Wanderlust acquisition, 20-40 employees
- `v2.2 tasks and above`: Keystone Mortgage Partners Universe - A residential mortgage brokerage in Charlotte, NC; 30 employees, 8 departments.

Heads up: each universe has its own fixed date

- MoveOps Universe - fixed date as April 26, 2026
- Keystone Mortgage Universe - fixed date as April 28, 2026

Your Assigned Persona

You will be assigned a default persona. Your tasks should make sense from this persona's perspective — Chloe wouldn't audit financials, Marcus wouldn't coordinate a client retreat, Samira wouldn't negotiate a sales deal.

If your assigned persona is "saturated" (you've exhausted the interesting situations for that role), you may switch to a different persona. When you do, mark which persona the task is written for so we can track the distribution.

The agent has access to ALL data regardless of persona — there are no tool or data restrictions. The persona shapes the perspective and the kind of tasks you create, not what the agent can see.

Personas

- In the universe summaries you will see them like this:

Name	Title	Location	Personality
Elena Rostova	CEO	San Francisco	Visionary, impatient, sends 3 AM emails, demands high-level summaries
Priya Chakrabarti	Executive Assistant	San Francisco	Elena's gatekeeper — organized, diplomatic, fiercely protective of Elena's time

Elena and Priya are central to the MoveOps universe.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

- In the persona briefs documents, you will have access to detailed information about each persona, like Brittany, from the Keystone universe:

Brittany Wallace -- Receptionist / Front Desk

Active work: *Brittany is the first voice callers hear and the first face visitors see. She handles call routing, visitor check-ins, mail distribution, and front-desk overflow. She also fields borrower calls that come in to the general line.*

Key relationships: *Internal: Grace (her direct report path), Jordan Blake (the subject of her complaint), all staff (she routes calls/visitors). External: walk-in borrowers, visiting realtors, delivery vendors.*

Open threads:

- *Harassment complaint filed (scenario_2b42ecf2): Brittany submitted a formal written complaint to Grace documenting Jordan Blake's repeated comments about her appearance, drink invitations after she said no, and an unwelcome shoulder touch. She prepared a timestamped written statement saved as a PDF. The investigation is in intake phase -- no interviews have occurred yet.*
- *Chaotic borrower (scenario_d8361499): A borrower on LN-2026-00621 is simultaneously texting, emailing, and calling multiple Keystone staff. Brittany is fielding some of these calls at the front desk.*

Recent activity: *Formal complaint email to Grace. Written statement PDF saved in* `documents/hr_notes/brittany_wallace/`*. Slack message noting the shoulder incident. Draft email documenting the incident while still fresh.*

Explore both universes and be familiarized with how they work and behave!

Important note: You don't need to write "I am Elena" in the prompt. The agent won't know what persona you're simulating — it has access to everything. The persona guides what tasks you create, not how the agent operates.

What Data Can the Agent Access?

The agent can access the company's email, Slack channels, Linear issues, calendar, contacts, CRM, and other business tools. For prompt writing, you do not need to specify the specific tool names or parameters (in rubrics writing, you will need to specify the tool names depending on the rubric category). What matters is that the data your task references actually exists in the universe.

Step 2: Explore and Edit the Universe

Explore

The MoveOps Universe

- **The MoveOps Universe Summary** (please read carefully)
- **MoveOps Personas**
- **A Reference Sheet with current Artifacts**
- **MoveOps Business Functions**

The Keystone Mortgage Partners Universe

- **Keystone Mortgage Partners - Universe Summary** (please read carefully)
- **Keystone Mortgage Partners - Persona Briefs**
- **Mortgage Broker - Scenario Storylines**
- **Keystone Mortgage Partners - Artifacts Reference Sheet**
- **Keystone Mortgage Partners - Business Functions**

Use the universe explorer and chat agent to understand what data exists:

- Search emails for interesting threads, complaints, follow-ups
- Browse Slack channels for discussions, incidents, celebrations
- Check Linear for open issues, projects, and their statuses
- Look at the CRM for client deals, pipeline, and account health
- Check contacts, Quickbooks, and other data

Here's a non-exhaustive list of what to look for:

- Someone made a mistake: Wrong data sent to wrong client, mixed-up entities, missed follow-up
- Incomplete work: Unapproved expenses, unfinished migration, open tickets nobody owns
- Conflicting information: Email says one thing, Slack says another, CRM shows a third
- Business pressure: Renewals coming up, board meetings, angry clients, cost overruns, production incidents
- Hidden root causes: The obvious explanation is wrong; the real cause is buried across multiple services

Edit

Here's a non-exhaustive list of what you can add:

- Email threads that create a new problem or situation
- Linear issues that the agent needs to discover
- CRM deal status changes that create urgency

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

- Slack messages with conflicting/confusing information
- Contact data that creates near-miss entity confusion (similar names)

Danger Zone: The chat agent for editing is currently unconstrained — it will make whatever changes you ask for without checking for conflicts. This means:

- It won't detect if your new data contradicts existing data (dates, names, relationships)
- It won't cascade changes — if you rename a person in email, their Slack/Linear/CRM data won't update automatically

You must manually review edits for:

- Universe consistency: Do dates, names, and facts align across services?
- Storyline consistency: Does the new data make sense in this universe story?
- Cross-service coherence: If you add an email about a flight, does the flight data exist too?

[NOTE]: Improved editing tooling is being developed (scenario generation, change logging). Detailed instructions will be provided in the doc.

Step 3: Write the Prompt

The Goal

Write a natural, open-ended work request that requires the agent to explore broadly across the universe data. The prompt should be solvable if the agent explores deeply enough, pays attention to detail, and reasons correctly, but we expect the agent should fail on at least some aspects. Your prompt should require the agent to perform 40+ tool calls to meet our complexity thresholds.

Good Prompts vs Bad Prompts

GOOD: Simple question, deep investigation required

"I'm reviewing our relocation coordinators and want to check how Fatimah Al-Rashidi is doing. Can you look at her recent work and tell me if everything looks good or if there are any concerns?"

Why it works: Simple ask, but answering it correctly requires checking emails, Linear tickets, CRM records, Slack messages across multiple clients — and the agent has to notice that Fatimah mixed up two Noahs with similar names (Noah Fitzgerald at GreenStack vs Noah Fitzpatrick at Axiom). Same apartment and same flight confirmation sent to both. The agent needs to cross-reference data from multiple sources to catch this.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

GOOD: Investigation + action

"The weather API keeps returning nulls and I'm getting paged. Something changed in the last few days. Figure out what's going on, update my Linear ticket with what you find, and if someone caused this, loop in Elena."

Why it works: Requires investigation across Linear, Slack, Notion, CRM. Root cause is Julian's unauthorized demo calling the API every 3 seconds. Agent must investigate then take write actions.

GOOD: Open-ended performance review

"How are our account managers doing? Look at their recent client interactions, response times, any complaints, and give me a report."

Why it works: Requires checking email threads for each AM, CRM deal progress, Slack conversations, customer complaints. Agent must synthesize across many data points.

BAD: Pre-solved

"Julian's demo is calling the weather API every 3 seconds causing rate limiting. Fix it."

Why it fails: Tells the agent the root cause. No investigation needed.

BAD: Command list

"Search emails from NovaCorp. Check their CRM status. Send them an apology email."

Why it fails: Tells the agent exactly what steps to take.

BAD: Bolted-together requests

"Check weather in Miami, update my calendar for next week, email Marcus about Q3, and look up Seattle flights."

Why it fails: Four unrelated requests stapled together. Each sub-request should causally flow from the same situation.

BAD: Not tool-dependent

"What's the best strategy for retaining enterprise clients?"

Why it fails: Answerable from general knowledge. Doesn't need MoveOps data.

BAD: Contrived difficulty

"Find the email from January 15th at exactly 3:47 PM that mentions a budget of $14,237.89, and cross-reference it with the third comment on Linear issue ENG-417."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Why it fails: Artificially specific. A real person would never phrase a request this way.

Core Rules for Prompt Writing

- Don't name tools or parameters — say "check what Ravi sent me," not "use search_emails"
- Don't pre-solve — the agent should investigate and discover
- Don't include internal IDs — say "the Pinnacle proposal," not "issue_pinnacle_proposal"
- Write naturally — like a real person talking to their assistant, not a command list
- One coherent situation — every sub-request should flow from the same problem
- The agent must fail — if it solves everything perfectly, the task isn't hard enough. Iterate.
- Not contrived — difficulty comes from real data complexity, not artificial constraints

NEW: The agent is now capable of inferring time. It will take **April 26, 2026,** as today's date, which means you are able to create relative time scenarios in prompts now.

- "*Zara sent me this email 3 days ago*" means the email was received April 23, 2026.
- "*... and schedule a meeting for next Friday*" is referring to May 1st, 2026.
- Fixed dates are also valid; just beware anything before April 26, 2026, is in the past, and after it is obviously in the future.

Prompt Patterns That Create Difficulty

Branching: The agent makes a decision based on what it finds, then takes different actions depending on the outcome.

"The April relocation wave is coming up fast and I have no idea which moves are actually ready to go. Can you check where things stand across the board and make sure our records reflect reality? Anything that's good to go should be marked accordingly, and anything with gaps needs to get flagged so the right coordinator picks it up before we're scrambling last minute."

Write tools: `airtable_update_records`, `linear_create_issue`

Step-by-step dependency: Figure out A before you can do B, and B before C.

"The DOT compliance audit flagged some gaps in our hazmat documentation and I need to make sure this gets handled before the June 5 deadline. Figure out which recent moves involved hazmat materials, check if the paperwork is complete, and make sure whoever is responsible for the gaps has a ticket assigned to them. If anyone on the team is already overloaded, loop in Chloe so she can rebalance."

Write tools: `linear_create_issue`, `conversations_add_message` (if escalating to Chloe)

Stacking: Multiple related asks unified by a single purpose. Stacking is NOT the same as bolting — stacked asks share a common context, while bolted requests are unrelated.

"I'm meeting Patricia at Canopy Health tomorrow and I need to walk in prepared. Make sure our internal tracking is up to date on their account, get me caught up on any open items the ops team is working on for them, and block out time with Catalina this afternoon so we can align before the call."

Write tools: `crm_create_engagement`, `linear_create_comment`, `calendar_add_calendar_event`

Further Improving Your Prompts

Investigation + Action: The agent must figure something out then do something about it.

"What's causing the cost spike? Once you figure it out, brief Elena and Marcus with what you found, and create a plan to fix it."

Implicit Requirements: Things the agent should obviously do that you don't explicitly say.

"Email the proposal to me and CC Elena." Implicit: Don't email the client directly before internal review. Stay within budget. Include actual details, not just "we're working on it."

Information Friction: The answer isn't all in one place.

"Who dropped the ball on the Axiom onboarding?"

The answer requires connecting: an email thread where the account manager promised a timeline, a Slack message where engineering flagged a delay, a CRM note showing the client complained, and a Linear ticket that was assigned but never started.

Constraints: Budget limits, policy requirements, approval needs.

"Find alternative accommodation for the Seattle relocation, but it has to be under $250/night, within 2 miles of the office, and pet-friendly — check if the employee has any pet requirements in their relocation file."

Step 3.5: Plan Your Oracle Events

Before running the agent, write down the Oracle Events (OEs) — the key steps a correct agent would take to solve your task. This is free-form text, not structured JSON. It proves you've thought through the solution and helps you write better rubrics later.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

For each key step, note:

- What action needs to happen (send email, create issue, look up contact)
- What information needs to be discovered
- What tool would be used and what parameters matter
- Whether it is a write action or a read/lookup action — this determines which rubric type applies

OEs describe steps that involve tool use — not what the final response should say.

How OEs Drive Rubric Writing

OEs feed into a two-filter workflow:

Filter 1 — OEs: From all tool calls in your OEs, identify write actions — these go into Outcome rubrics (1.1 + 1.2). Identify key facts the user asked to be told directly — these go into Outcome rubrics (2.1). Then identify every remaining non-write-action tool call. These are your candidates for process rubrics.

Filter 2 — Matrix: For each non-write-action candidate, apply the Process Rubric Decision Matrix (in the Rubrics Guidelines) to decide whether to write TS, QC, both, or neither.

Write actions always go into Outcome. Process rubrics (TS/QC) are only for read/lookup actions not already covered by Outcome.

Example Oracle Events

Prompt: *"Send an email to Chloe from Fatimah about a relocation proposal"*

OE 1: Look up Fatimah's and Chloe's contact details using contacts_search_contacts or contacts_get_contacts.

> *→ Read/lookup action. Both contacts_search_contacts and contacts_get_contacts are default tools → skip TS for either. Query is obvious ('Fatimah', 'Chloe') → skip QC.*

OE 2: Send email from fatimah.al-rashidi@moveops.com to chloe.vance@moveops.com with subject and body about a relocation proposal.

> *→ Write action → goes into Outcome. No TS or QC for send_email — Outcome already covers it.*
>
> *→ Outcome 1.1: "An email was sent (via send_email) from fatimah.al-rashidi@moveops.com to chloe.vance@moveops.com."*

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

→ Outcome 1.2: "The email subject relates to a relocation proposal (e.g., 'Relocation Proposal' or similar)."

Step 4: Run the Agent and Iterate

The Workflow

1. Submit your prompt and the agent runs (expect 30+ minutes)
2. While the agent runs, start thinking about rubrics based on your Oracle Events (see Rubrics Guidelines)
3. After the agent finishes, review the trajectory — what did the agent do? What did it miss?
4. If the agent solved everything perfectly: your task isn't hard enough. Options:
    - Make the prompt more open-ended so the agent explores more broadly
    - Add more data to the universe that creates confusion or complexity
    - Ask for more things in the prompt (stacking)
    - Introduce conflicting information across services
5. If the agent failed: Good! Write rubrics that capture both what it got right and what it got wrong.
6. Iterate until you have a task where the agent fails on some rubric criteria.

Tip: Use Haiku for quick iteration while designing the task (faster, cheaper). Once you're satisfied with the prompt and universe setup, switch to Opus 4.6 for the final 6 runs that determine pass@1.

Difficulty Target

We run the agent 6 times with Opus 4.6. Your task is ready to submit when:

- Average 40+ tool calls
- All 6 fail is fine (pass is 0%)
- At most 2 runs pass (pass@1 ≤ 40%, proves the task is hard enough)

If all 6 runs pass: the task is too easy. Iterate.

Where AIs Might Fail

- Two entities have similar names and the agent confuses them
- The obvious answer is wrong and the real answer requires deeper digging
- Information is scattered across many services and the agent doesn't find all of it
- The agent makes correct observations but draws the wrong conclusion
- The agent takes some actions correctly but misses others

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Step 5: Write the Rubrics

See the Rubrics Guidelines for full instructions. Quick summary:

Three Categories

- **Tool Selection (TS)** — did the agent use the right tool?
- **Query Construction (QC)** — did it pass the right parameters to critical tool calls?
- **Outcome** — was the result correct?

Core Rule: Outcome First

- Write all Outcome rubrics first: write actions go into 1.1/1.2, key facts reported to the user go into 2.1. Then add TS and QC only for what Outcome doesn't already cover.
- Outcome rubrics are the top priority — they are the most reliable training signal
- Aim for 50/50 to 80/20 outcome-to-process split (TS + QC = process)
- If Outcome already confirms a tool was called with the right parameters, do NOT write a TS or QC rubric for that tool

Outcome Sub-Categories (3, not 5)

- **1.1** Write-action results — did the right action happen with the right details? (verified from trajectory)
- **1.2** Action content — does the content match what was needed? (verified from trajectory parameters)
- **2.1** Key facts/findings — only when the final response is the deliverable (pure investigation tasks)

Do NOT write:

- Grounding checks — not required. Write Outcome rubrics based on what the agent reports and does.
- Confirmation of actions in the final response — if 1.1 passes, the action happened. No separate confirmation needed.

Process Rubric Decision Matrix

For each non-write-action tool call from your OEs, apply this matrix:

Situation	TS?	QC?

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Non-default tool + query is obvious	✅ Yes	❌ No
Non-default tool + bad query could miss data	✅ Yes	✅ Yes
Default tool + query is non-trivial	❌ No	✅ Yes
Default tool + any keyword finds data	❌ No	❌ No

In the MoveOps Universe

Default tools (always skip TS): `search_emails`, `conversations_search_messages`, `conversations_history`, `contacts_get_contacts`, `contacts_search_contacts`

Non-default tools (write TS if agent might skip): `airtable_*`, `quickbooks_*`, `crm_*`, `calendar_*`, `linear_*`

In the Keystone Mortgage Universe

Default tools (always skip TS): `email*, conversations_* (slack), contacts_*`

Non-default tools (write TS if agent might skip): `mortgage_los_*, stripe_*, crm_*, filesystem_*, quickbooks_*`

WARNING: As of April 10, 2026 the FileSystem tool isn't ready yet. It will be possible to inspect PDFs, Docs and other artifacts with this tool in the near future.

- **For now, do not use it**

Other Rubric Rules

- Each rubric has three fields: criterion + justification + evidence
- Criterion is what the judge evaluates; justification and evidence are for QC reviewers only
- QC rubrics use Exact Match (EM) for structured fields (email addresses, dates) and Fuzzy for freetext fields — include "(or similar)" for fuzzy values
- Scoring: A run passes only if ALL rubrics pass (any single rubric failure = run failure). Target: pass@1 ≤ 40%.
- Every rubric must be: specific, self-contained, objective, verifiable from the trajectory
- Subjective language is banned in rubric criteria ("enough," "professional," "thorough," "helpful," "good")

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Step 6: Verify Before Submitting

- Data exists: Every fact the task references exists in the universe (verified in explorer)
- Broad exploration: Task requires extensive tool calls across 3+ services
- Tool-dependent: Cannot be answered from general knowledge alone
- No tool names: Prompt doesn't mention MCP tools or parameter names
- No pre-solving: Prompt doesn't tell the agent the answer
- No internal IDs: No database IDs unless a real person would know them
- Natural language: Reads like a real person, not a command list
- Not bolted: Every request causally connects to the same situation
- Not contrived: Difficulty comes from real data complexity, not artificial constraints
- Agent fails: The agent does NOT pass all rubric criteria on at least one run
- Universe edits consistent: Any changes you made are consistent with existing data
- Rubrics written: Outcome first — write actions covered by 1.1/1.2, investigation by 2.1, process rubrics only for gaps via the decision matrix

---

Common Mistakes

Mistake 1: Agent Solves It Perfectly If the agent passes all rubric criteria, the task is too easy. Add complexity to the universe, make the prompt more open-ended, or ask for more things.

Mistake 2: Command Lists Instead of Tasks "Search emails. Check CRM. Send apology." — This is a script. State the goal: "NovaCorp is upset. Figure out the full picture and handle it."

Mistake 3: Data Doesn't Exist You write about a client meeting that isn't in the calendar, or reference an email thread that doesn't exist. Always verify in the explorer.

Mistake 4: Contrived Difficulty Adding weird formatting constraints, demanding exact numerical precision, or creating unrealistic scenarios to trick the model. Difficulty should come from the natural complexity of the business situation.

Examples of contrived (BAD):

- "Respond in exactly 3 sentences using only passive voice" — format constraint, not business complexity
- "Find the email from January 15th at 3:47 PM" — unrealistic precision, no real person remembers exact timestamps
- "Intentionally send an incorrect email so we can test the error handling" — asking the agent to do something a real employee wouldn't do

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z

Examples of natural difficulty (GOOD):

- Two clients with similar names (Noah Fitzgerald vs Noah Fitzpatrick) causing a mix-up
- An unauthorized demo by Julian that's buried in a CRM note, not in any incident ticket
- A cost spike whose root cause requires connecting a Slack celebration from 2 weeks ago to a CRM contract signed the same week

Mistake 5: Bolted-Together Requests Four unrelated tasks crammed together. Every sub-request should flow from the same situation.

Mistake 6: Pre-Solved "Julian's demo causes rate limiting" — tells the agent the answer. Say instead: "The API keeps failing. Figure out why."

Mistake 7: Too Short / Too Simple "Rebook the retreat" — too terse. Real work requests come with context (what's happening), constraints (budget, timing), and multiple asks (email this person, notify that person, track it in Linear).

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.986Z